I have a PyTorch model called MyktbekModel that takes a batch of activity sequences and returns two outputs: activity logits and remaining time. I need a training function for it. How do I write a basic training loop in PyTorch?

LISA Pro

Рассуждение заняло 4 секунд(ы)

Basic PyTorch Training Loop for MyktbekModel

Here's a complete training function tailored to your model's dual outputs (activity logits and remaining time):

Basic Training Function

python Свернуть Запустить Копировать

```python
import torch
import torch.nn as nn
from torch.utils.data import DataLoader

def train_epoch(model, dataloader, optimizer, activity_criterion, time_criterion,
                device, alpha=0.5):
    """
    Train for one epoch.

    Args:
        model: Your MyktbekModel
        dataloader: DataLoader with activity sequences
        optimizer: Optimization algorithm (e.g., Adam)
        activity_criterion: Loss for activity logits (e.g., CrossEntropyLoss)
        time_criterion: Loss for remaining time (e.g., MSELoss)
        device: 'cuda' or 'cpu'
        alpha: Weight balancing between activity and time losses (default: 0.5)

    Returns:
        Average loss for the epoch
    """
    model.train()
    total_loss = 0.0
```

```
    for batch_idx, (sequences, activity_labels, time_labels) in enumerate(dataloader):
        # Move data to device
        sequences = sequences.to(device)
        activity_labels = activity_labels.to(device)
        time_labels = time_labels.to(device)

        # Zero gradients
        optimizer.zero_grad()

        # Forward pass
        activity_logits, remaining_time = model(sequences)

        # Compute losses
        activity_loss = activity_criterion(activity_logits, activity_labels)
        time_loss = time_criterion(remaining_time, time_labels)

        # Combined loss
        loss = alpha * activity_loss + (1 - alpha) * time_loss

        # Backward pass
        loss.backward()

        # Update weights
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(dataloader)
    return avg_loss
```

Complete Training Loop

python Свернуть Запустить Копировать

```
def train_model(model, train_loader, val_loader, num_epochs, learning_rate, device):
    """
    Full training loop with validation.
    """
    # Initialize optimizers and loss functions
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    activity_criterion = nn.CrossEntropyLoss()  # For activity classification
```

```
    time_criterion = nn.MSELoss()  # For remaining time regression

    # Track best model
    best_val_loss = float('inf')

    for epoch in range(num_epochs):
        # Training
        train_loss = train_epoch(
            model, train_loader, optimizer,
            activity_criterion, time_criterion, device
        )

        # Validation
        val_loss = validate_epoch(
            model, val_loader,
            activity_criterion, time_criterion, device
        )

        print(f"Epoch {epoch+1}/{num_epochs}")
        print(f"  Train Loss: {train_loss:.4f}")
        print(f"  Val Loss: {val_loss:.4f}")

        # Save best model
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            torch.save(model.state_dict(), 'best_model.pth')

    return model
```

Validation Function

python Свернуть Запустить Копировать

```
@torch.no_grad()
def validate_epoch(model, dataloader, activity_criterion, time_criterion, device):
    """Evaluate model on validation set."""
    model.eval()
    total_loss = 0.0

    for sequences, activity_labels, time_labels in dataloader:
        sequences = sequences.to(device)
        activity_labels = activity_labels.to(device)
```

```
        time_labels = time_labels.to(device)

        activity_logits, remaining_time = model(sequences)

        activity_loss = activity_criterion(activity_logits, activity_labels)
        time_loss = time_criterion(remaining_time, time_labels)

        loss = 0.5 * activity_loss + 0.5 * time_loss
        total_loss += loss.item()

    return total_loss / len(dataloader)
```

Usage Example

python Свернуть Запустить Копировать

```
# Setup
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = MyktbekModel().to(device)

# DataLoaders (you'll need to create these)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

# Train
trained_model = train_model(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    num_epochs=50,
    learning_rate=0.001,
    device=device
)
```

Key Points

COMPONENT	PURPOSE
`model.train()`	Sets model to training mode (enables dropout, batch norm updates)
`optimizer.zero_grad()`	Clears previous gradients before each batch
`loss.backward()`	Computes gradients via backpropagation
`optimizer.step()`	Updates model weights
`@torch.no_grad()`	Disables gradient computation for validation (saves memory)

`model.eval()`	Sets model to evaluation mode (disables dropout, etc.)

Tips for Multi-Output Models

1. **Balance losses**: Adjust `alpha` based on which task is more important
2. **Normalize scales**: Ensure both losses are on similar scales
3. **Monitor separately**: Track activity_loss and time_loss individually during training
4. **Learning rate scheduling**: Consider using `ReduceLROnPlateau` based on validation loss

Would you like me to elaborate on any specific part, such as learning rate schedulers, gradient clipping, or custom dataset creation?

The training function needs to match exactly how our project's DataLoader works. It yields batches of (x, y_activity, y_time) where y_time shape is (batch,) not (batch, 1). The function should be called train_myktbek_model and take these arguments: model, train_loader, num_epochs, learning_rate, device. It should move the model to device, use Adam optimizer, CrossEntropyLoss for activity and MSELoss for time. If y_time has shape (batch,) unsqueeze it to (batch, 1) before MSELoss. Print average loss per epoch and return list of per-epoch losses.

LISA Pro

Рассуждение заняло 7 секунд(ы)

python Свернуть Запустить Копировать

```python
import torch
import torch.nn as nn

def train_myktbek_model(model, train_loader, num_epochs, learning_rate, device):
    """
    Train MyktbekModel with activity and time outputs.

    Args:
        model: MyktbekModel instance
        train_loader: DataLoader yielding (x, y_activity, y_time) batches
        num_epochs: Number of training epochs
        learning_rate: Learning rate for Adam optimizer
        device: 'cuda' or 'cpu'

    Returns:
        List of average losses per epoch
    """
```

```
"""

# Move model to device
model.to(device)

# Initialize optimizer and loss functions
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
activity_criterion = nn.CrossEntropyLoss()
time_criterion = nn.MSELoss()

# Track losses per epoch
epoch_losses = []

for epoch in range(num_epochs):
    model.train()
    total_loss = 0.0
    num_batches = 0

    for x, y_activity, y_time in train_loader:
        # Move data to device
        x = x.to(device)
        y_activity = y_activity.to(device)
        y_time = y_time.to(device)

        # Unsqueeze y_time from (batch,) to (batch, 1) for MSELoss
        if y_time.dim() == 1:
            y_time = y_time.unsqueeze(1)

        # Zero gradients
        optimizer.zero_grad()

        # Forward pass
        activity_logits, remaining_time = model(x)

        # Compute losses
        activity_loss = activity_criterion(activity_logits, y_activity)
        time_loss = time_criterion(remaining_time, y_time)

        # Combined loss
        loss = activity_loss + time_loss

        # Backward pass
```

```
            loss.backward()

            # Update weights
            optimizer.step()

            total_loss += loss.item()
            num_batches += 1

        # Calculate average loss for epoch
        avg_loss = total_loss / num_batches
        epoch_losses.append(avg_loss)

        print(f"Epoch {epoch + 1}/{num_epochs}, Average Loss: {avg_loss:.4f}")

    return epoch_losses
```

Usage Example

python Свернуть Запустить Копировать

```
# Setup
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = MyktbekModel()

# Train
losses = train_myktbek_model(
    model=model,
    train_loader=train_loader,
    num_epochs=50,
    learning_rate=0.001,
    device=device
)
```

Key Features

REQUIREMENT	IMPLEMENTATION
Function name	`train_myktbek_model`
Arguments	Exactly as specified
Batch format	`(x, y_activity, y_time)`
y_time reshape	`unsqueeze(1)` if dim == 1
Optimizer	Adam

Activity loss	CrossEntropyLoss
Time loss	MSELoss
Output	Prints per-epoch loss, returns list